

12 — Regola o dato?

La domanda a cui risponde: la stessa informazione — “questa carta si può giocare a un torneo” — in parte si calcola e in parte si scarica. Come si decide quale metà è quale? E perché sbagliare la divisione produce software che invecchia male anche quando è scritto benissimo?

Il fatto

Serviva un filtro “carte valide nei tornei”. Il Pokémon TCG ha due formati ufficiali, uno dentro l'altro:

- **Standard** — solo le carte recenti. Ogni aprile la *rotazione* ne butta fuori un anno intero.
- **Expanded** — dal Nero e Bianco (2011) in poi, meno una lista di carte **bandite** perché troppo forti.

Sembra un dato solo. Sono due cose di natura diversa, e la differenza si vede solo chiedendosi: **se aspetto un anno senza toccare niente, cosa smette di essere vero?**

Informazione	Cambia?	Dove sta
Il marchio stampato su una carta (H)	mai	data/legalita.json, scaricato
Quali marchi sono legali oggi (H, I, J)	ogni aprile	MARCHI_STANDARD, in src/data/legalita.js
Se una carta è bandita in Expanded	ogni tanto, senza preavviso	data/legalita.json, scaricato

Il *marchio di regolamentazione* è la letterina dentro il quadratino in basso a sinistra della carta. È inchiostro: non cambierà mai. Quali lettere valgano, invece, è una decisione che qualcuno prende a Tokyo una volta l'anno.

Se avessi salvato direttamente `legale: true`, come pure l'API offriva (`legal.standard`), il file sarebbe stato **giusto il giorno del download e sbagliato per sempre dopo**, senza che nulla lo segnalasse. Salvando il marchio e tenendo la regola nel codice, la rotazione del 2027 costa una lettera in una costante:

```
// src/data/legalita.js
export const MARCHI_STANDARD = Object.freeze(['H', 'I', 'J']);
```

La regola generale

Scarica ciò che **non puoi dedurre**. Calcola ciò che **puoi scrivere come regola**. Quando i due si sovrappongono, scarica il dato più stabile dei due.

L'Expanded è l'altra metà, ed è il caso opposto: sembra deducibile (“dal Nero e Bianco in poi”) e non lo è. Verificando contro l'API, la regola per serie sbagliava in tre modi diversi:

- **38 carte bandite** dentro set per il resto legalissimi: Archeops, Ghecis, Shaymin EX, la Foresta delle Piante Giganti, l'Asso di Elisio, la Tipaccia. Non c'è nessuna proprietà della carta da cui dedurlo — è una lista scritta a mano da un ufficio;
- **set interi recenti ma esclusi**: Pokémon TCG Pocket (è un gioco digitale), le promo McDonald's, i Kit Allenatore;
- e all'inverso, le **Energie base** di set antichissimi che restano legali sempre, in qualunque formato.

Una lista arbitraria non si indovina: si scarica. Il codice che ci prova non è elegante, è solo sbagliato con più passaggi.

L'eccezione: quando il dato scaricato è peggio della regola

Le Energie base sono legali sempre e ovunque — è una riga del regolamento. TCGdex però ne dà per Standard solo 178 su 204, lasciando fuori quelle dei Kit Allenatore e di qualche set vecchio. Sono carte **identiche**: è disattenzione nei dati, non una regola.

Qui il codice del progetto disobbedisce di proposito alla sorgente:

```
// src/data/legalita.js
function eEnergiaBase(carta) {
  return carta?.categoria === 'Energia' && carta?.tipoEnergia === 'Base';
}

export function formatoDi(carta) {
  if (!carta) return null;
  if (eEnergiaBase(carta)) return PER_CODICE.get('standard');
  // ...
}
```

Il criterio per decidere se disobbedire: **conosco la regola vera e so perché la sorgente diverge?** Qui sì (sono ristampe identiche non riclassificate). Sui banditi no — e infatti lì si obbedisce.

Un'API interrogata bene costa 8 richieste invece di 21.000

Il marchio non era nei file locali: `tools/scarica-set.mjs` normalizza le carte tenendo solo i campi che servivano al catalogo. Ottenere una carta per carta significava 21.037 richieste, dieci minuti buoni.

Ma TCGdex, come molte API REST, filtra l'elenco per campo:

```
GET /v2/it/cards?regulationMark=H → 1.301 carte, una risposta
GET /v2/it/cards?legal.expanded=true → 13.764 carte, una risposta
```

Sette marchi più l'Expanded: **otto richieste** per l'intero catalogo mondiale. La lezione non è su TCGdex — è che prima di scrivere un ciclo su ventimila elementi conviene passare mezz'ora a leggere cosa sa fare la sorgente. Il ciclo avrebbe funzionato; sarebbe stato semplicemente 2.600 volte più lento.

Il risultato si comprime scrivendo il valore **dominante** per set e solo le deroghe carta per carta:

```
"marchi": { "sv09": "I", "sv08": { "_": "H", "252": "G" } }
"espansi": { "sv08": true, "xy6": { "_": true, "077": false } }
```

11 KB invece di ~300. `sv09` è tutto marchio I; in `sv08` una sola carta su 252 fa eccezione; `xy6-077` è Shaymin EX, che il suo set legale l'ha ma è bandita.

Dove si applica: il punto unico di ingresso

Il file scaricato deve arrivare sulle carte. Il progetto aveva già lo stesso problema con `data/ristampe.json`, e la soluzione si ripete:

```
// src/data/dataset.js
export async function caricaSet(idSet) {
  if (cacheSet.has(idSet)) return cacheSet.get(idSet);
  const set = await leggiJson(`_${idSet}.json`);
  set.carte = await completaRistampe(idSet, set.carte ?? []);
  set.carte = await applicaLegalita(idSet, set.carte);
  cacheSet.set(idSet, set);
  return set;
}
```

Al caricamento del set, una volta sola. Non nel componente che mostra la pastiglia, non nel filtro: lì bisognerebbe ricordarsene ogni volta, e chi scrive il prossimo componente non se ne ricorderà. In termini Angular è la differenza fra arricchire i dati in un `resolver` e farlo in ogni template con una pipe: la pipe funziona finché qualcuno non dimentica di metterla.

Perché non direttamente dentro i file di `data/set/?` Perché sono due dati che cambiano — la lista dei banditi si allunga, i marchi di un set nuovo arrivano dopo — mentre i 6,4 MB dei set si scaricano una volta e restano lì. **La roba che cambia sta in un file piccolo, da sola.**

Il campo che vale false di proposito

Dettaglio piccolo con conseguenze grosse. applicaLegalita scrive espansa **sempre**, anche quando è false:

```
return {
  ...carta,
  marchio: valorePerCarta(marchiSet, numero) ?? null,
  espansa: valorePerCarta(espansiSet, numero) === true,
};
```

Serve a distinguere tre stati che il codice ingenuo confonde in due:

Stato	Come si riconosce	Cosa risponde formatoDi
carta legale	espansa === true	Standard o Expanded
carta illegale	espansa === false	Fuori formato
carta mai timbrata	espansa === undefined	null — “non lo so”

Il terzo caso è reale: un set che non si è riusciti a leggere, o legalita.json assente da una cache incompleta. Rispondere “fuori formato” sarebbe una **bugia dall’aria autorevole**, e la conseguenza sarebbe una carta sparita da un filtro senza spiegazione. È lo stesso ragionamento di 06-misurare-un-mazzo.md: zero e “non lo so” non sono lo stesso numero.

E il filtro se ne ricorda:

```
// src/ui/griglia-collezione/raggruppa.js
if (formato && formatoDi(carta)?.codice !== formato) return false;
```

Un null non combacia con nessun codice, quindi la carta esce dai risultati di “solo le Standard” — che è la scelta giusta: chi chiede quell’elenco vuole fidarsene, e un forse in mezzo alle carte valide è peggio di una carta in meno.

Il no che non si scrive

Il chip del formato compare solo su Standard ed Expanded:

```
const chipFormato =
  formato && formato.codice !== 'fuori'
    ? `<span class="chip chip-formato" data-formato="${formato.codice}" ...>`
    : '';
```

Non è pigrizia: in una collezione di famiglia le carte fuori formato sono la maggioranza, e un’etichetta grigia su quasi ogni carta scrive “questa non vale” sotto tutta la collezione. Chi cerca proprio quelle ha il filtro. **Un’interfaccia non deve rispondere alle domande che nessuno ha fatto** — soprattutto quando la risposta è deprimente.

Le costanti che documentano la loro scadenza

MARCHI_STANDARD cambia una volta l’anno. Chi la troverà fra dieci mesi non saprà da dove uscivano quelle tre lettere, quindi il commento porta la data e il modo in cui sono state verificate:

```
/**
 * **Da aggiornare a ogni rotazione**, tipicamente in aprile [...]
 *
 * Verificato il 28/07/2026 contro `legal.standard` di TCGdex: H, I e J
 * coprono esattamente le 3.182 carte che l'API dà per Standard, senza
 * eccezioni in nessuno dei due sensi.
 */
```

Una costante con una scadenza e nessuna nota è una bomba a orologeria educata.

Domande di verifica

1. `data/legalita.json` contiene `"sv08": { "_": "H", "252": "G" }`. Cosa risponde `valorePerCarta` per la carta 004? E per la 252? Perché la chiave `_` non può collidere con un numero di collezione vero?
2. Immagina che la rotazione di aprile 2027 tolga il marchio H. Elenca **tutti** i file da toccare. Ora immagina la stessa cosa se avessimo salvato `legal.standard` invece del marchio: quanti sono, e quanto durano?
3. `formatoDi({ nome: 'Pikachu' })` torna `null`, non `'fuori'`. Scrivi un caso d'uso concreto in cui la differenza si vede a schermo, e uno in cui non si vede affatto. Perché vale la pena distinguerli anche nel secondo caso?
4. Le Energie base sono trattate con una regola scritta a mano *contro* il dato scaricato. Trova nel progetto un altro punto in cui si disobbedisce alla sorgente (suggerimento: `completaRistampe` in `src/data/dataset.js`) e di' in che cosa i due casi si somigliano e in che cosa no.